Low-Rank Spectral Fingerprints for AI-Generated Music Detection

Interim report 1: scoping memo

Iryna Matvieieva Petro Mozil

Linear Algebra, UCU Data Science Master Programme, November 4, 2026

Domain	Audio forensics / music provenance
Method family	STFT magnitude spectrograms; exact and randomized SVD
LA object	For each 10-s mono clip, $M \in \mathbb{R}_{\geq 0}^{1025 \times T}$, with $T \approx 431$; analyse the leading $k = 20$ singular values
Algorithms	Own randomized SVD ($k = 20$, oversampling $p = 10$, one power step); exact NumPy/LAPACK SVD as reference
Data	FakeMusicCaps (subset), paired with retrievable original MusicCaps clips [**?**]

1 Problem and motivation

The detection of AI-generated music has proven a problem for platforms, rights holders and listeners. In this work, we intend to distinguish AI-generated audio from human-generated audio. Given a short audio clip, we predict human-origin versus text-to-music-generated. We do so by checking whether the low-rank structure of its STFT magnitude spectrogram supports this distinction on held-out clips, rather than create a universal detector. Prior work also warns that strong controlled results may not generalize across manipulations or datasets [**?**, **?**], so we try to balance classes and group each original/synthetic caption pair in the same data split.

Success is held-out balanced accuracy ≥ 0.65, with the lower bound of a 95% confidence interval above 0.50. Separately, randomized SVD should achieve relative ℓ_2 error $\leq 10^{-2}$ in the first 20 normalized squared singular-value features on at least 95% of test clips, relative to exact SVD. If the classification threshold is not met, we should deem these features insufficient in this setup.

2 Linear-algebra formulation

Each clip is resampled to 22.05 kHz. A centred STFT (window/FFT length 2048, hop 512) gives a nonnegative magnitude matrix $M \in \mathbb{R}_{\geq 0}^{F \times T}$, where $F = 1025$ and $T \approx 431$ for ten seconds. Its singular value decomposition is

$$M = U \Sigma V^\top, \qquad \sigma_1 \geq \sigma_2 \geq \cdots \geq 0. \tag{1}$$

We use $z = (p_1, \ldots, p_{20})$ as features for logistic regression, where $p_i = \sigma_i^2 / \|M\|_F^2$ and $\|M\|_F^2 = \sum_i \sigma_i^2$. The features summarize the distribution of spectrogram energy across orthogonal components.

The LA task is to compute or approximate the leading singular values and test whether this low-rank summary preserves class information. Because the features discard phase and temporal ordering, useful cues may be lost even with an accurate decomposition.

3 Methods

3.1 Exact thin SVD

A standard dense SVD gives the comparison values and the optimal rank-k approximation for the Frobenius norm by the Eckart–Young–Mirsky theorem.

For $F \geq T$, its cost is $O(FT^2 + T^3)$ flops and $O(FT)$ storage. It is accurate and straightforward, but repeats work for every clip. In our case, we will have moderate matrix sizes, and so it may already be fast, limiting the practical speedup available to a randomized method. Small singular values can also be sensitive to rounding, and an accurate decomposition does not guarantee useful detection features.

NumPy's np.linalg.svd uses LAPACK's _gesdd routine to calculate SVD. _gesdd implements a divide-and-conquer algorithm on a bidiagonal (diagonal with entries non-zero above or below main diagonal) matrix. It divides the matrix into sizes that are convenient for QR iteration for SVD calculation, and then accumulates the results.

We left out the algorithm pseudo-code for the _gesdd implementation for brevity.

Cost. Phase 1 requires $O(FT^2 - \frac{1}{3}T^3)$ flops for Householder bidiagonalization. Phase 2 (divide-and-conquer) and Phase 3 (vector accumulation) require $O(T^3)$ flops. Total arithmetic cost is $O(FT^2 + T^3)$, but the divide-and-conquer conquering step heavily benefits from parallel matrix-matrix operations, making it much faster in practice than sequential QR iteration for large dimensions.

Listing 1: Singular Value Decomposition in NumPy.

```
import numpy as np

def householder_dnc_thin_svd(A):
    U, s, Vt = np.linalg.svd(A)
    return U, s, Vt
```

3.2 Randomized SVD

Following the randomized range-finding framework of Halko, Martinsson, and Tropp [**?**], the core idea of randomized SVD is to capture the essential column space (range) of a matrix $A \in \mathbb{R}^{F\times T}$ using a low-dimensional random projection before performing an exact factorization. The procedure unfolds in four main stages:

We wish to find a low-rank approximation of rank k. Instead of examining all T dimensions, we draw a random test matrix $\mathbf{\Omega} \in \mathbb{R}^{T\times \ell}$ whose entries are independent and identically distributed standard Gaussian variables, $\mathbf{\Omega}_{ij} \sim \mathcal{N}(0, 1)$.

The sketch dimension ℓ is chosen slightly larger than the target rank k via an oversampling parameter p (e.g., $k = 20$, $p = 10$, yielding $\ell = 30$).

Multiplying A by $\mathbf{\Omega}$ forms the sketch matrix:

$$Y = A\mathbf{\Omega} \in \mathbb{R}^{F\times \ell} \tag{2}$$

If the singular values of A decay slowly, the random sketch alone may capture too much noise from the tail of the spectrum. To sharpen the separation between the dominant components and the trailing noise, we apply $q = 1$ power iteration steps (subspace iteration). This alternates multiplications by A and $A^\top$ to damp out lesser singular values:

$$\tilde{Y} = A^\top Q, \qquad \text{QR factorization: } \tilde{Y} = \tilde{Q}\tilde{R} \tag{3}$$

$$Y = A\tilde{Q}, \qquad \text{QR factorization: } Y = QR \tag{4}$$

This accelerates the spectral decay rate without requiring a full matrix factorization.

Once the refined orthonormal basis Q is established, we project the original matrix A onto this lower-dimensional subspace to form a much smaller matrix B:

$$B = Q^\top A \in \mathbb{R}^{\ell\times T} \tag{5}$$

Because $\ell \ll T$, computing the exact SVD of B via standard routines (e.g., LAPACK's *_gesdd*) is relatively fast.

$$B = \tilde{U}\mathbf{s}V^\top \tag{6}$$

Finally, we map the left singular vectors back to the original space by setting $U = Q\tilde{U}$, and truncate all factors to the target rank k.

Cost. With q power steps, the overall flop count is $O((2q + 1)FT\ell + (F + T)\ell^2)$, and memory requirement is bounded by $O(FT + (F + T)\ell)$. Because $\ell \ll T$, the dense SVD phase runs in $O(T\ell^2)$ time instead of $O(T^3)$.

Algorithm 1: Randomized SVD with Power Step

Input: Matrix $A \in \mathbb{R}^{F \times T}$ $(F \geq T)$, target rank k, oversampling parameter p, power iterations q

Output: Rank-k SVD factors: $U_k \in \mathbb{R}^{F \times k}$, $\mathbf{s}_k \in \mathbb{R}^k$, $V_k \in \mathbb{R}^{T \times k}$

$\ell \leftarrow k + p$; // Total sketch dimension, $\ell \approx 30$

Draw Gaussian random matrix $\mathbf{\Omega} \in \mathbb{R}^{T \times \ell}$ where $\mathbf{\Omega}_{ij} \sim \mathcal{N}(0, 1)$;

$Y \leftarrow A\mathbf{\Omega}$; // Form initial sketch: $O(FT\ell)$ flops

; // Power iteration to accelerate singular value decay

for $i = 1$ *to* q **do**

  $Q, _ \leftarrow \mathrm{qr}(Y)$;

  $\tilde{Y} \leftarrow A^\top Q$;

  $\tilde{Q}, _ \leftarrow \mathrm{qr}(\tilde{Y})$;

  $Y \leftarrow A\tilde{Q}$;

$Q, _ \leftarrow \mathrm{qr}(Y)$; // Orthonormal basis for range of A: $O(F\ell^2)$ flops

; // Project and perform small SVD

$B \leftarrow Q^\top A$; // Compress matrix to dimension $\ell \times T$: $O(FT\ell)$ flops

$\tilde{U}, \mathbf{s}, V^\top \leftarrow \mathrm{SVD}(B)$; // Exact SVD on reduced matrix: $O(T\ell^2)$ flops

; // Reconstruct full left singular vectors and truncate to rank k

$U \leftarrow Q\tilde{U}$;

$U_k \leftarrow U[:, 1:k]$; $\mathbf{s}_k \leftarrow \mathbf{s}[1:k]$; $V_k \leftarrow V[:, 1:k]$;

return U_k, $\mathbf{s}_k$, V_k

Listing 2: Randomized SVD with $q = 1$ power step in NumPy.

```python
import numpy as np

def randomized_svd(A, k=20, p=10, q=1):
    F, T = A.shape
    ell = k + p

    # Draw Gaussian random test matrix
    Omega = np.random.normal(size=(T, ell))
    Y = A @ Omega

    # Power iteration (q steps)
    for _ in range(q):
        Q, _ = np.linalg.qr(Y)
        Y_tilde = A.T @ Q
        Q_tilde, _ = np.linalg.qr(Y_tilde)
        Y = A @ Q_tilde

    # Final orthogonalization for the range space
    Q, _ = np.linalg.qr(Y)

    # Project A into smaller subspace and compute exact SVD
    B = Q.T @ A
    U_tilde, s, Vt = np.linalg.svd(B, full_matrices=False)

    # Reconstruct original left singular vectors
    U = Q @ U_tilde

    # Truncate to target rank k
    return U[:, :k], s[:k], Vt[:k, :].T
```

3.3 Cost and failure modes

Randomized SVD is a low-rank approximation of thin SVD. It only computes the singular value for a random subspace of a matrix, and might be vulnerable to flat spectra (ones, where $|\sigma_{k+1}/\sigma_k| \approx 1$). Unlike regular SVD, randomized SVD does not rely on reduction to the bidiagonal form, rather, the bulk of the computation is the matrix multiplication during the power iteration step.

Exact SVD requires storing intermediate orthogonal transformation matrices and workspaces that scale with T^2 and randomized SVD operates primarily on compressed sketch matrices of size $\ell = k + p$ (where $\ell \ll T$), and so it requires substantially less memory.

Randomized SVD does not handle ill-conditioned matrices well. Should the spectrum of a matrix decay slowly, the power iteration step would not capture the matrice‘s spectrum in the randomized subspace well.

4 Implementation and experiments

4.1 Setup

The project will be implemented in Python using NumPy and SciPy for linear algebra and signal processing, with scikit-learn for the baseline classifier and evaluation metrics.

Audio preprocessing should use a common pipeline: conversion to mono at 16 kHz, low-pass filtering at 7 kHz, loudness normalisation, and a common MP3 encode/decode step.

The datasets are FakeMusicCaps and the corresponding human-made MusicCaps clips, paired by their original captions. We will balance the classes and split the data by caption to prevent related examples from appearing in both training and test sets.

For each clip, we need to compute a log-magnitude STFT spectrogram and extract singular-value features, including entropy-based effective rank and energy concentration. We also compute principal angles between the leading left singular subspaces of consecutive spectrogram windows. A simple bandwidth-only classifier will serve as a diagnostic baseline for detecting pipeline-related differences. The current implementation is being developed and tested on small subsets before the full dataset is processed.

4.2 Results

The initial experiments will compare the proposed features with the bandwidth-only baseline using balanced accuracy and ROC-AUC on a held-out test set. We will also investigate the relationship between the singular-value spectrum and temporal subspace stability, and whether these features generalise across different generated clips.

Exact SVD is expected to be inexpensive for individual spectrograms, so it will be used as the reference method for per-clip features. Randomized SVD will instead be applied to a larger matrix formed by stacking spectrogram frames from training clips, allowing us to learn a shared spectral basis.

We will report quantitative results on real audio after we retrieve and process the datasets.

5 Discussion and limitations

The main risk is that AI-generated and human-made clips may come from different audio pipelines. Differences in sampling rate, YouTube compression, bandwidth, mastering, or loudness could create spectral cues unrelated to the process that generated the music. Resampling, common low-pass filtering, loudness normalisation, and common codec processing should reduce some of these differences, but cannot guarantee their removal. The bandwidth-only baseline will help reveal whether simple technical artifacts already predict the class labels.

A second limitation is that low-rank spectral structure is not necessarily unique to AI-generated music. Human recordings and generated music can both contain strong redundancy, while different generators may produce different spectral characteristics. For this, principal angles between consecutive

windows should provide a concrete measure of subspace change, but, for classification, they might not be of much use.

6 Conclusion

Guidance. *What you achieved, the main difficulties, and what you would do next. Half a page is enough.*

7 Plan and division of work

Guidance. *Interim reports only; this section disappears from the final report.* Interim report 1*: the planned steps and who does what.* Interim report 2*: what is not working yet, any change of plan, and the outline of the final report.*

References

[1] L. Comanducci, P. Bestagini and S. Tubaro, "FakeMusicCaps: A Dataset for Detection and Attribution of Synthetic Music Generated via Text-to-Music Models," *Journal of Imaging*, vol. 11, no. 7, article 242, 2025. doi:10.3390/jimaging11070242.

[2] D. Afchar, G. Meseguer-Brocal and R. Hennequin, "AI-Generated Music Detection and its Challenges," in *Proc. IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2025. doi:10.1109/ICASSP49660.2025.10890655.

[3] Y. Li, Q. Sun, H. Li, L. Specia and B. W. Schuller, "Explainable detection of machine generated music and early systematic evaluation," *Scientific Reports*, vol. 16, article 13757, 2026. doi:10.1038/s41598-026-42133-7.

[4] N. Halko, P. G. Martinsson and J. A. Tropp, "Finding Structure with Randomness: Probabilistic Algorithms for Constructing Approximate Matrix Decompositions," *SIAM Review*, vol. 53, no. 2, pp. 217–288, 2011. doi:10.1137/090771806.